



Node.js 学习计划

互联网大厂 P6 高级工程师技术路线



总周期：24周（6个月）



每周投入：15-20小时



目标级别：P6高级工程师



8大核心模块



12个实战项目



100+面试题解析

目 录

学习路径总览与时间规划

学习路径图
每周时间分配建议
P6技能评估标准

第一章 基础理论与环境搭建（第1-3周）

1.1 Node.js核心概念与架构
1.2 事件循环机制深入解析
1.3 异步编程模型
1.4 开发环境配置与工具链
1.5 推荐学习资源

第二章 核心技术深入（第4-6周）

2.1 模块化系统
2.2 文件系统操作
2.3 流(Streams)处理
2.4 Buffer与字符编码

第三章 网络编程（第7-9周）

3.1 HTTP/HTTPS协议实现
3.2 RESTful API设计与开发
3.3 WebSocket应用开发
3.4 TCP/UDP网络编程

第四章 数据库交互（第10-12周）

--

4.1 关系型数据库操作

4.2 NoSQL数据库应用

4.3 ORM框架使用

第五章 性能优化 (第13-15周)

5.1 内存管理与泄漏排查

5.2 CPU密集型任务处理

5.3 集群模式应用

5.4 缓存策略设计

第六章 工程化实践 (第16-18周)

6.1 代码规范与质量控制

6.2 单元测试与集成测试

6.3 CI/CD流程搭建

6.4 Docker容器化部署

第七章 框架与生态 (第19-21周)

7.1 Express框架深入学习

7.2 Koa框架深入学习

7.3 NestJS框架深入学习

7.4 中间件开发与微服务架构

第八章 高级主题 (第22-24周)

8.1 安全最佳实践

8.2 分布式系统设计

8.3 服务监控与日志系统

8.4 Node.js与前端工程结合

实践项目清单

常见面试题解析

推荐学习资源汇总



学习路径总览与时间规划

学习路径图

以下为从零基础到P6高级工程师的完整学习路径，共24周，分为8个核心模块，每个模块3周。建议按顺序学习，后期模块依赖前期基础。



每周时间分配建议

活动类型	时间占比	每周时长	说明
理论学习	30%	5-6小时	阅读文档、书籍、观看教程视频
编码实践	40%	6-8小时	跟随示例编码、完成练习题
项目实战	20%	3-4小时	完成模块对应实战项目
复习总结	10%	1.5-2小时	整理笔记、撰写博客、面试题练习

24周详细学习计划

第1周 | Node.js核心概念与安装配置

- ✅ 安装Node.js LTS版本，配置nvm版本管理
- ✅ 理解Node.js架构：V8 + libuv + C++ Bindings
- ✅ 掌握REPL交互式环境使用
- ✅ 了解Node.js与浏览器JavaScript的核心区别
- ✅ 编写第一个Node.js程序并运行

第2周 | 事件循环与异步编程基础

- ☒ 深入学习事件循环6个阶段及执行顺序
- ☒ 理解microtask与macrotask优先级
- ☒ 掌握回调函数模式及回调地狱问题
- ☒ 学习Promise基本用法与链式调用
- ☒ 练习分析异步代码执行顺序

第3周 | 异步编程进阶与开发工具链

- ☒ 精通async/await语法与错误处理
- ☒ 了解Generator函数与异步迭代
- ☒ 配置VS Code + ESLint + Prettier开发环境
- ☒ 掌握npm/yarn/pnpm包管理器使用
- ☒ 完成模块1实战项目：异步任务调度器

第4周 | 模块化系统深入

- ☒ 深入理解CommonJS模块加载机制与缓存
- ☒ 掌握ES Modules配置与使用
- ☒ 理解两种模块系统的差异与互操作
- ☒ 了解循环依赖的处理方式
- ☒ 学习package.json配置与语义化版本

第5周 | 文件系统与Buffer

- ☒ 掌握fs模块同步/异步/Promise API
- ☒ 理解文件描述符、权限与文件监听
- ☒ 掌握Buffer创建、操作与字符编码转换
- ☒ 了解Buffer与TypedArray的关系
- ☒ 实现文件批量处理工具

第6周 | 流(Streams)处理

- ☒ 掌握四种流类型：Readable/Writable/Duplex/Transform
- ☒ 理解流动模式与暂停模式
- ☒ 掌握pipeline管道操作与错误处理
- ☒ 自定义Transform流实现数据转换
- ☒ 完成模块2实战项目：大文件日志分析器

第7周 | HTTP服务器与客户端

- ☒ 掌握http/https模块创建服务器
- ☒ 理解HTTP请求/响应生命周期
- ☒ 掌握HTTPS证书配置
- ☒ 了解HTTP/2在Node.js中的实现
- ☒ 实现静态文件服务器

第8周 | RESTful API设计

- ☒ 掌握RESTful API设计原则与规范
- ☒ 理解HTTP方法语义与状态码
- ☒ 实现请求验证、错误处理与版本管理
- ☒ 了解OpenAPI/Swagger文档规范
- ☒ 设计并实现完整的CRUD API

第9周 | WebSocket与TCP/UDP

- ☒ 掌握ws库实现WebSocket服务器
- ☒ 实现实时聊天室应用
- ☒ 掌握net模块创建TCP服务器/客户端
- ☒ 理解TCP粘包问题与解决方案
- ☒ 完成模块3实战项目：实时协作白板

第10周 | 关系型数据库

- ☒ 掌握MySQL2/pg驱动使用与连接池配置
- ☒ 理解事务处理与隔离级别
- ☒ 掌握SQL注入防护（预处理语句）
- ☒ 实现数据库连接池管理模块
- ☒ 学习数据库索引优化基础

第11周 | NoSQL数据库

- ☒ 掌握MongoDB CRUD操作与聚合管道
- ☒ 掌握Redis数据结构与应用场景
- ☒ 实现Cache-Aside缓存策略
- ☒ 理解Redis发布/订阅模式
- ☒ 实现分布式锁

第12周 | ORM框架

- ☒ 掌握Sequelize模型定义与关联关系
- ☒ 掌握TypeORM装饰器与Repository模式
- ☒ 理解N+1查询问题与预加载
- ☒ 掌握数据库迁移(Migration)管理
- ☒ 完成模块4实战项目：电商订单系统后端

第13周 | 内存管理

- ☒ 理解V8内存管理与垃圾回收机制
- ☒ 掌握内存泄漏的常见原因与排查方法
- ☒ 使用heapdump/v8-profiler进行内存分析
- ☒ 了解WeakRef/FinalizationRegistry
- ☒ 实现内存监控中间件

第14周 | CPU优化与Worker Threads

- ☒ 理解Node.js单线程对CPU密集型任务的影响
- ☒ 掌握worker_threads多线程方案
- ☒ 了解child_process多进程方案
- ☒ 实现任务队列与分发策略
- ☒ 完成图片批量处理服务

第15周 | 集群与缓存策略

- ☒ 掌握Cluster模块实现多进程
- ☒ 掌握PM2进程管理工具
- ☒ 理解缓存穿透/击穿/雪崩及解决方案
- ☒ 实现多级缓存架构
- ☒ 完成模块5实战项目：高并发短链服务

第16周 | 代码规范与质量控制

- ☒ 配置ESLint + Prettier + TypeScript
- ☒ 掌握Husky + lint-staged Git钩子
- ☒ 理解Commitlint提交规范
- ☒ 学习代码审查(Code Review)最佳实践
- ☒ 为已有项目配置完整的代码规范体系

第17周 | 单元测试与集成测试

- ☒ 掌握Jest测试框架使用
- ☒ 理解测试金字塔与测试策略
- ☒ 掌握Mock/Stub/Spy技巧
- ☒ 了解Supertest进行API集成测试
- ☒ 为项目编写测试，覆盖率达到80%+

第18周 | CI/CD与Docker

- ☒ 掌握GitHub Actions / GitLab CI配置
- ☒ 掌握Dockerfile编写与多阶段构建
- ☒ 掌握docker-compose服务编排
- ☒ 搭建完整的CI/CD流水线
- ☒ 完成模块6实战项目：全自动化部署的API服务

第19周 | Express框架

- ☒ 深入理解Express中间件机制（洋葱模型）
- ☒ 掌握路由、错误处理与请求生命周期
- ☒ 实现自定义中间件（认证、日志、限流）
- ☒ 了解Express性能优化与安全配置
- ☒ 使用Express构建完整RESTful API

第20周 | Koa框架

- ☒ 理解Koa的async/await中间件模型
- ☒ 掌握Koa核心：context/request/response
- ☒ 对比Express与Koa的设计哲学差异
- ☒ 实现Koa自定义中间件
- ☒ 使用Koa重构API服务

第21周 | NestJS与微服务

- ☒ 掌握NestJS模块/控制器/服务架构
- ☒ 理解依赖注入(DI)与控制反转(IoC)
- ☒ 掌握NestJS守卫/拦截器/管道
- ☒ 了解微服务架构设计与通信模式
- ☒ 完成模块7实战项目：微服务架构博客平台

第22周 | 安全最佳实践

- ☒ 掌握JWT/OAuth2.0认证授权实现
- ☒ 理解XSS/CSRF/SQL注入/SSRF等攻击防护
- ☒ 掌握helmet/cors/rate-limit等安全中间件
- ☒ 了解加密算法与密钥管理
- ☒ 实现完整的安全认证体系

第23周 | 分布式系统与监控

- ☒ 理解CAP定理与分布式一致性
- ☒ 掌握消息队列(RabbitMQ/Kafka)应用
- ☒ 实现分布式链路追踪
- ☒ 掌握Winston/Pino日志系统
- ☒ 配置Prometheus + Grafana监控

第24周 | 前端工程结合与综合实战

- ☒ 掌握SSR(Next.js/Nuxt.js)原理与实现
- ☒ 了解BFF(Backend For Frontend)架构
- ☒ 掌握Webpack/Vite与Node.js的结合
- ☒ 完成模块8实战项目：全栈电商平台
- ☒ 全面复习，准备模拟面试

P6技能评估标准

达到P6级别需满足以下技能要求：

Node.js核心原理

深入理解事件循环、V8引擎、libuv



异步编程

精通Promise/async-await/Generator



网络编程

独立设计RESTful API、WebSocket应用



数据库

熟练使用SQL/NoSQL及ORM框架



性能优化

能排查内存泄漏、设计缓存策略



工程化

CI/CD、Docker、测试体系搭建



框架应用

系统设计

精通至少一个主流框架，了解微服务



分布式设计、安全防护、监控体系



1 基础理论与环境搭建（第1-3周）

1.1 Node.js核心概念与架构

学习目标

- 理解Node.js的设计理念与适用场景
- 掌握Node.js架构组成：V8引擎 + libuv + C++ Bindings
- 了解Node.js与浏览器JavaScript的区别
- 掌握Node.js的版本管理策略（LTS vs Current）

核心知识点

Node.js是基于Chrome V8引擎的JavaScript运行时，采用单线程事件驱动架构。其核心架构由以下层次组成：

层次	组件	职责
应用层	JavaScript代码	业务逻辑编写
绑定层	C++ Bindings	桥接JS与底层C++库
引擎层	V8 Engine	JavaScript编译执行
平台层	libuv	异步I/O、事件循环
系统层	操作系统API	文件、网络等系统调用

核心要点

Node.js并非真正的单线程——V8引擎和libuv内部都使用了线程池。所谓“单线程”是指JavaScript代码的执行是单线程的，I/O操作则由libuv的线程池和操作系统异步机制处理。

1.2 事件循环机制深入解析

学习目标

- 掌握事件循环的6个阶段及其执行顺序
- 理解microtask与macrotask的优先级
- 能分析复杂异步代码的执行顺序

事件循环6个阶段



关键代码示例

```
console.log('1 - 同步代码开始');

setTimeout(() => {
  console.log('2 - setTimeout 宏任务');
}, 0);

setImmediate(() => {
  console.log('3 - setImmediate 宏任务');
});

Promise.resolve().then(() => {
  console.log('4 - Promise 微任务');
});

process.nextTick(() => {
  console.log('5 - nextTick 微任务 (优先级最高)');
});

console.log('6 - 同步代码结束');

// 输出顺序: 1 → 6 → 5 → 4 → 2/3 (2和3顺序不确定)
```

面试考点

Q: process.nextTick 与 Promise.then 的区别?

A: process.nextTick 属于微任务队列，但优先级高于 Promise 微任务。nextTick 在当前操作完成后立即执行，Promise.then 在微任务队列的下一批执行。大量 nextTick 可能导致 I/O 饥饿。

1.3 异步编程模型

学习目标

- 掌握回调函数模式及其问题（回调地狱）
- 精通Promise链式调用与错误处理
- 熟练使用async/await语法糖
- 了解Generator函数与异步迭代

异步编程演进

阶段	模式	优点	缺点
1.0	回调函数	简单直接	回调地狱、错误处理困难
2.0	Promise	链式调用、统一错误处理	语义不够直观
3.0	Generator + co	同步写法	需要执行器、不够原生
4.0	async/await	同步语义、原生支持	需注意并发控制

async/await最佳实践

```
class UserService {
  async getUserById(id) {
    try {
      const user = await UserModel.findById(id);
      if (!user) {
        throw new NotFoundError('用户不存在');
      }
      return user;
    } catch (error) {
      logger.error('获取用户失败', { id, error: error.message });
      throw error;
    }
  }

  async getUsersByIds(ids) {
    const promises = ids.map(id => this.getUserById(id));
    return Promise.allSettled(promises);
  }

  async createUser(data) {
    const existing = await UserModel.findOne({ email: data.email });
    if (existing) {
      throw new ConflictError('邮箱已注册');
    }
    return UserModel.create(data);
  }
}
```

💡 实践建议

1. 始终对async函数进行try/catch错误处理
2. 使用Promise.allSettled替代Promise.all处理批量请求
3. 避免在循环中使用await，改用Promise.all并发
4. 使用AbortController实现可取消的异步操作

1.4 开发环境配置与工具链

学习目标

- 掌握nvm/n进行Node.js版本管理
- 配置ESLint + Prettier代码规范
- 掌握npm/yarn/pnpm包管理器使用
- 配置VS Code高效开发环境

推荐工具链

类别	推荐工具	用途
版本管理	nvm-windows / n	管理多个Node.js版本
包管理器	pnpm (推荐) / yarn	依赖管理、monorepo支持
代码规范	ESLint + Prettier	代码检查与格式化
类型检查	TypeScript	静态类型系统
调试工具	VS Code Debugger / ndb	断点调试
API测试	Postman / HTTPie	接口调试
进程管理	pm2 / nodemon	开发/生产进程管理

1.5 推荐学习资源

Node.js官方文档

<https://nodejs.org/docs/latest/api/>

最权威的参考文档，建议通读核心模块API

Node.js官网

<https://nodejs.org/>

Node.js官方网站，包含下载、文档、社区资源

Node.js Best Practices

<https://github.com/goldbergonyi/nodebestpractices>

社区最佳实践合集，涵盖架构、安全、性能等

《Node.js设计模式（第3版）》

Mario Casciaroma著，深入设计模式与架构，P6必读

《深入浅出Node.js》

朴灵著，中文经典，深入底层原理



Node.js Official YouTube

<https://www.youtube.com/@Nodejs>

官方频道，包含Node.js Conf演讲视频

2 核心技术深入（第4-6周）

2.1 模块化系统

学习目标

- 深入理解CommonJS模块加载机制
- 掌握ES Modules的使用与配置
- 理解两种模块系统的差异与互操作
- 了解模块缓存机制与循环依赖处理

CommonJS vs ES Modules对比

特性	CommonJS	ES Modules
语法	require() / module.exports	import / export
加载方式	运行时加载（同步）	编译时静态分析（异步）
输出	值的拷贝	值的引用（绑定）
this指向	module.exports	undefined
循环依赖	返回已执行部分的快照	引用绑定，可能获取未初始化值
Tree Shaking	不支持	支持
启用方式	默认（.js）	package.json "type": "module" 或 .mjs

CommonJS模块缓存机制

```
const cache = require.cache;

function requireUncached(modulePath) {
  delete cache[require.resolve(modulePath)];
  return require(modulePath);
}

function inspectModuleCache() {
  const modules = Object.keys(cache);
  console.log(`当前缓存了 ${modules.length} 个模块`);
  modules.forEach(m => {
    if (!m.includes('node_modules')) {
      console.log(` - ${m}`);
    }
  });
}
```

面试考点

Q: require的加载过程?

1. 解析模块路径 (相对/绝对/内置/node_modules)
2. 检查缓存, 有则直接返回module.exports
3. 无缓存则创建新module对象
4. 根据扩展名(.js/.json/.node)用不同方式加载
5. 编译执行, 将exports返回并缓存

2.2 文件系统操作

学习目标

- 掌握fs模块的同步/异步/Promise API
- 理解文件描述符与权限管理
- 掌握文件监听 (watch) 机制
- 了解fs.promises API的最佳实践

fs.promises最佳实践

```
const fs = require('fs').promises;
const path = require('path');

async function ensureDir(dirPath) {
  try {
    await fs.access(dirPath);
  } catch {
    await fs.mkdir(dirPath, { recursive: true });
  }
}

async function safeReadFile(filePath, encoding = 'utf-8') {
  try {
    const content = await fs.readFile(filePath, encoding);
    return content;
  } catch (error) {
    if (error.code === 'ENOENT') {
      return null;
    }
    throw error;
  }
}

async function writeJsonFile(filePath, data) {
  const dir = path.dirname(filePath);
  await ensureDir(dir);
  await fs.writeFile(filePath, JSON.stringify(data, null, 2), 'utf-8');
}
```

2.3 流(Streams)处理

学习目标

- 掌握四种基本流类型：Readable、Writable、Duplex、Transform
- 理解流的两种模式：流动模式(flowing)与暂停模式(paused)
- 掌握pipeline管道操作与错误处理
- 能自定义Transform流实现数据处理

流的四种类型

类型	描述	典型示例
Readable	可读流，数据的来源	fs.createReadStream, HTTP请求
Writable	可写流，数据的目的地	fs.createWriteStream, HTTP响应
Duplex	双工流，可读可写（独立）	net.Socket, TLS套接字
Transform	变换流，读入数据变换后输出	zlib.createGzip, crypto流

自定义Transform流示例

```

const { Transform } = require('stream');

class JsonLineTransform extends Transform {
  constructor(options = {}) {
    super({ ...options, objectMode: true });
    this.buffer = '';
  }

  _transform(chunk, encoding, callback) {
    this.buffer += chunk.toString();
    const lines = this.buffer.split('\n');
    this.buffer = lines.pop();

    for (const line of lines) {
      if (line.trim()) {
        try {
          const obj = JSON.parse(line);
          this.push(obj);
        } catch (e) {
          callback(new Error(`JSON解析失败: ${line}`));
          return;
        }
      }
    }
    callback();
  }

  _flush(callback) {
    if (this.buffer.trim()) {
      try {
        this.push(JSON.parse(this.buffer));
      } catch (e) {
        callback(new Error(`JSON解析失败: ${this.buffer}`));
        return;
      }
    }
    callback();
  }
}

const { pipeline } = require('stream/promises');
const fs = require('fs');

async function processLargeJsonFile(inputPath, outputPath) {
  await pipeline(
    fs.createReadStream(inputPath),
    new JsonLineTransform(),
    async function* (source) {
      for await (const obj of source) {
        yield JSON.stringify({ ...obj, processed: true }) + '\n';
      }
    },
    fs.createWriteStream(outputPath)
  );
}

```

🔥 核心要点

始终使用pipeline替代pipe方法！ pipe不会正确传播错误，可能导致内存泄漏。pipeline能确保流结束时正确清理资源，并统一处理错误。

2.4 Buffer与字符编码

学习目标

- 理解Buffer的内存分配机制（Slab分配）
- 掌握字符编码转换（utf8/ascii/base64/hex）
- 了解Buffer与TypedArray的关系
- 掌握Buffer的性能优化技巧

Buffer核心操作

```
const buf = Buffer.alloc(256);
const strBuf = Buffer.from('Hello Node.js', 'utf-8');

console.log(strBuf.toString('hex'));
console.log(strBuf.toString('base64'));

const combined = Buffer.concat([Buffer.from('Hello '), Buffer.from('World')]);

const sliced = combined.subarray(0, 5);
sliced[0] = 74;
console.log(combined.toString());

const base64Buf = Buffer.from('SGVsbG8gV29ybGQ=', 'base64');
console.log(base64Buf.toString('utf-8'));

function compareBuffers(a, b) {
  return Buffer.compare(a, b);
}
```

📖 Node.js Stream官方文档

<https://nodejs.org/docs/latest/api/stream.html>

流的完整API参考，包含所有事件和方法说明

📖 Node.js Buffer官方文档

<https://nodejs.org/docs/latest/api/buffer.html>

Buffer API完整参考

📖 Node.js FS官方文档

<https://nodejs.org/docs/latest/api/fs.html>

文件系统模块完整API参考

3 网络编程（第7-9周）

3.1 HTTP/HTTPS协议实现

学习目标

- 掌握http/https模块创建服务器与客户端
- 理解HTTP请求/响应生命周期
- 掌握HTTPS证书配置
- 了解HTTP/2在Node.js中的实现

HTTP服务器核心代码

```
const http = require('http');
const https = require('https');
const fs = require('fs');

const httpApp = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ message: 'HTTP Server' }));
});

const httpsOptions = {
  key: fs.readFileSync('./cert/key.pem'),
  cert: fs.readFileSync('./cert/cert.pem'),
};

const httpsApp = https.createServer(httpsOptions, (req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ message: 'HTTPS Server' }));
});

httpApp.listen(80, () => console.log('HTTP on :80'));
httpsApp.listen(443, () => console.log('HTTPS on :443'));
```

3.2 RESTful API设计与开发

学习目标

- 掌握RESTful API设计原则与规范
- 理解HTTP方法语义（GET/POST/PUT/PATCH/DELETE）
- 掌握请求验证、错误处理与版本管理
- 了解OpenAPI/Swagger文档规范

RESTful API设计规范

HTTP方法	路径	功能	是否幂等
GET	/api/users	获取用户列表	是
GET	/api/users/:id	获取单个用户	是
POST	/api/users	创建用户	否
PUT	/api/users/:id	全量更新用户	是
PATCH	/api/users/:id	部分更新用户	否
DELETE	/api/users/:id	删除用户	是

统一响应格式设计

```
class ApiResponse {
  static success(data, message = 'success') {
    return {
      code: 0,
      message,
      data,
      timestamp: Date.now(),
    };
  }

  static error(code, message, details = null) {
    return {
      code,
      message,
      details,
      timestamp: Date.now(),
    };
  }

  static paginated(data, total, page, pageSize) {
    return {
      code: 0,
      message: 'success',
      data,
      pagination: {
        total,
        page,
        pageSize,
        totalPages: Math.ceil(total / pageSize),
      },
      timestamp: Date.now(),
    };
  }
}
```

3.3 WebSocket应用开发

学习目标

- 理解WebSocket协议与HTTP长轮询的区别

- 掌握ws库的使用
- 实现实时聊天、推送等应用
- 了解Socket.IO的封装与使用场景

WebSocket服务器实现

```
const WebSocket = require('ws');

const wss = new WebSocket.Server({ port: 8080 });

const rooms = new Map();

wss.on('connection', (ws, req) => {
  const userId = extractUserId(req);

  ws.on('message', (data) => {
    const msg = JSON.parse(data);
    switch (msg.type) {
      case 'join':
        joinRoom(ws, msg.roomId, userId);
        break;
      case 'chat':
        broadcastToRoom(msg.roomId, {
          type: 'chat',
          from: userId,
          content: msg.content,
          time: Date.now(),
        }, ws);
        break;
      case 'leave':
        leaveRoom(msg.roomId, userId);
        break;
    }
  });

  ws.on('close', () => {
    cleanupUser(userId);
  });
});

function broadcastToRoom(roomId, message, excludeWs) {
  const room = rooms.get(roomId);
  if (!room) return;
  for (const client of room) {
    if (client !== excludeWs && client.readyState === WebSocket.OPEN) {
      client.send(JSON.stringify(message));
    }
  }
}
```

3.4 TCP/UDP网络编程

学习目标

- 掌握net模块创建TCP服务器/客户端
- 掌握dgram模块实现UDP通信

- 理解TCP粘包问题与解决方案
- 了解Socket连接状态管理

TCP服务器与粘包处理

```
const net = require('net');

class PacketParser {
  constructor() {
    this.buffer = Buffer.alloc(0);
  }

  parse(data) {
    this.buffer = Buffer.concat([this.buffer, data]);
    const packets = [];

    while (this.buffer.length >= 4) {
      const length = this.buffer.readUInt32BE(0);
      if (this.buffer.length < 4 + length) break;

      const payload = this.buffer.subarray(4, 4 + length);
      this.buffer = this.buffer.subarray(4 + length);
      packets.push(JSON.parse(payload.toString('utf-8')));
    }

    return packets;
  }
}

const server = net.createServer((socket) => {
  const parser = new PacketParser();

  socket.on('data', (data) => {
    const packets = parser.parse(data);
    for (const packet of packets) {
      handlePacket(socket, packet);
    }
  });

  socket.on('error', (err) => {
    console.error('Socket error:', err.message);
  });
});

server.listen(3000, () => console.log('TCP Server on :3000'));
```

Node.js HTTP官方文档

<https://nodejs.org/docs/latest/api/http.html>

HTTP模块完整API参考

Node.js Net官方文档

<https://nodejs.org/docs/latest/api/net.html>

TCP/IPC网络模块完整API参考

ws库文档

<https://github.com/websockets/ws>

最流行的Node.js WebSocket实现

Socket.IO官方文档

<https://socket.io/docs/v4/>

支持回退、房间、命名空间的WebSocket框架

4 数据库交互 (第10-12周)

4.1 关系型数据库操作

学习目标

- 掌握MySQL/PostgreSQL的Node.js驱动使用
- 理解连接池管理与配置优化
- 掌握事务处理与隔离级别
- 了解SQL注入防护

MySQL连接池与事务

```

const mysql = require('mysql2/promise');

const pool = mysql.createPool({
  host: 'localhost',
  user: 'root',
  password: 'password',
  database: 'app_db',
  waitForConnections: true,
  connectionLimit: 10,
  queueLimit: 0,
  enableKeepAlive: true,
});

async function transferBalance(fromId, toId, amount) {
  const conn = await pool.getConnection();
  try {
    await conn.beginTransaction();

    const [from] = await conn.execute(
      'SELECT balance FROM accounts WHERE id = ? FOR UPDATE',
      [fromId]
    );

    if (from[0].balance < amount) {
      throw new Error('余额不足');
    }

    await conn.execute(
      'UPDATE accounts SET balance = balance - ? WHERE id = ?',
      [amount, fromId]
    );

    await conn.execute(
      'UPDATE accounts SET balance = balance + ? WHERE id = ?',
      [amount, toId]
    );

    await conn.commit();
  } catch (error) {
    await conn.rollback();
    throw error;
  } finally {
    conn.release();
  }
}

```

PostgreSQL使用pg库

```
const { Pool } = require('pg');

const pool = new Pool({
  connectionString: process.env.DATABASE_URL,
  max: 20,
  idleTimeoutMillis: 30000,
  connectionTimeoutMillis: 2000,
});

async function findUserByEmail(email) {
  const { rows } = await pool.query(
    'SELECT * FROM users WHERE email = $1',
    [email]
  );
  return rows[0] || null;
}
```

4.2 NoSQL数据库应用

学习目标

- 掌握MongoDB的CRUD操作与聚合管道
- 掌握Redis的数据结构与应用场景
- 理解MongoDB的索引优化
- 掌握Redis缓存策略 (Cache-Aside/Write-Through)

MongoDB聚合管道示例

```
const { MongoClient } = require('mongodb');

async function getUserOrderStats(db) {
  return db.collection('orders').aggregate([
    { $match: { status: 'completed', createdAt: { $gte: new Date('2024-01-01') } } },
    { $group: {
      _id: '$userId',
      totalOrders: { $sum: 1 },
      totalAmount: { $sum: '$amount' },
      avgAmount: { $avg: '$amount' },
    } },
    { $sort: { totalAmount: -1 } },
    { $limit: 100 },
    { $lookup: {
      from: 'users',
      localField: '_id',
      foreignField: '_id',
      as: 'user',
    } },
    { $unwind: '$user' },
    { $project: {
      userId: '$_id',
      username: '$user.name',
      totalOrders: 1,
      totalAmount: 1,
      avgAmount: { $round: ['$avgAmount', 2] },
    } },
  ]).toArray();
}
```

Redis缓存策略实现

```

const Redis = require('ioredis');

const redis = new Redis({
  host: 'localhost',
  port: 6379,
  retryStrategy(times) {
    return Math.min(times * 50, 2000);
  },
});

class CacheService {
  constructor(prefix = 'app', defaultTTL = 3600) {
    this.prefix = prefix;
    this.defaultTTL = defaultTTL;
  }

  _key(key) {
    return `${this.prefix}:${key}`;
  }

  async get(key) {
    const data = await redis.get(this._key(key));
    return data ? JSON.parse(data) : null;
  }

  async set(key, value, ttl = this.defaultTTL) {
    await redis.set(this._key(key), JSON.stringify(value), 'EX', ttl);
  }

  async getOrSet(key, fetchFn, ttl = this.defaultTTL) {
    const cached = await this.get(key);
    if (cached !== null) return cached;

    const data = await fetchFn();
    await this.set(key, data, ttl);
    return data;
  }

  async invalidate(pattern) {
    const keys = await redis.keys(this._key(pattern));
    if (keys.length > 0) {
      await redis.del(...keys);
    }
  }
}

```

4.3 ORM框架使用

学习目标

- 掌握Sequelize模型定义与关联关系
- 掌握TypeORM装饰器与Repository模式
- 理解N+1查询问题与预加载(eager loading)
- 掌握数据库迁移(Migration)管理

TypeORM实体定义

```
import { Entity, PrimaryGeneratedColumn, Column, OneToMany, CreateDateColumn, UpdateDateColumn } from 'typeorm';
import { Order } from './Order';

@Entity('users')
export class User {
  @PrimaryGeneratedColumn('uuid')
  id: string;

  @Column({ unique: true })
  email: string;

  @Column()
  name: string;

  @Column({ select: false })
  passwordHash: string;

  @Column({ default: 'active' })
  status: string;

  @OneToMany(() => Order, order => order.user)
  orders: Order[];

  @CreateDateColumn()
  createdAt: Date;

  @UpdateDateColumn()
  updatedAt: Date;
}
```

MySQL2官方文档

<https://github.com/sidorares/node-mysql2>

Node.js MySQL客户端，支持Promise和预处理语句

PostgreSQL pg文档

<https://node-postgres.com/>

PostgreSQL Node.js客户端完整文档

MongoDB Node.js Driver

<https://www.mongodb.com/docs/drivers/node/current/>

MongoDB官方Node.js驱动文档

Redis ioredis文档

<https://github.com/redis/ioredis>

功能最全的Node.js Redis客户端

TypeORM官方文档

<https://typeorm.io/>

TypeScript ORM框架，支持多种数据库

Sequelize官方文档

<https://sequelize.org/>

最流行的Node.js ORM，支持PostgreSQL/MySQL等

5 性能优化（第13-15周）

5.1 内存管理与泄漏排查

学习目标

- 理解V8内存管理与垃圾回收机制
- 掌握内存泄漏的常见原因与排查方法
- 熟练使用heapdump/v8-profiler等工具
- 了解WeakRef/FinalizationRegistry的使用

V8垃圾回收策略

算法	适用区域	特点
Scavenge（副GC）	新生代（Young Generation）	速度快、空间小、频繁执行
Mark-Sweep（主GC）	老生代（Old Generation）	标记清除、停顿较长
Mark-Compact	老生代	标记整理、解决内存碎片
Incremental Marking	老生代	增量标记、减少停顿

内存泄漏排查

```
const v8 = require('v8');
const fs = require('fs');

function takeHeapSnapshot(label) {
  const fileName = `heapdump-${label}-${Date.now()}.heapsnapshot`;
  const stream = v8.getHeapSnapshot();
  const file = fs.createWriteStream(fileName);
  stream.pipe(file);
  console.log(`Heap snapshot saved: ${fileName}`);
}

setInterval(() => {
  const usage = process.memoryUsage();
  console.log({
    rss: `${(usage.rss / 1024 / 1024).toFixed(2)} MB`,
    heapTotal: `${(usage.heapTotal / 1024 / 1024).toFixed(2)} MB`,
    heapUsed: `${(usage.heapUsed / 1024 / 1024).toFixed(2)} MB`,
    external: `${(usage.external / 1024 / 1024).toFixed(2)} MB`,
  });
}, 60000);
```

⚠ 常见内存泄漏场景

1. 全局变量持续增长（如缓存无上限）
2. 闭包引用未释放的大对象
3. 事件监听器未移除（EventEmitter泄漏）
4. 定时器未清理
5. 数据库连接/文件描述符未关闭

5.2 CPU密集型任务处理

学习目标

- 理解Node.js单线程对CPU密集型任务的影响
- 掌握worker_threads多线程方案
- 了解child_process多进程方案
- 掌握任务队列与分发策略

Worker Threads使用

```
const { Worker, isMainThread, parentPort, workerData } = require('worker_threads');

if (isMainThread) {
  function runHeavyTask(data) {
    return new Promise((resolve, reject) => {
      const worker = new Worker(__filename, { workerData: data });
      worker.on('message', resolve);
      worker.on('error', reject);
      worker.on('exit', (code) => {
        if (code !== 0) reject(new Error(`Worker stopped with exit code ${code}`));
      });
    });
  }

  async function processBatch(items) {
    const cpuCount = require('os').cpus().length;
    const chunkSize = Math.ceil(items.length / cpuCount);
    const chunks = [];

    for (let i = 0; i < items.length; i += chunkSize) {
      chunks.push(items.slice(i, i + chunkSize));
    }

    const results = await Promise.all(chunks.map(chunk => runHeavyTask(chunk)));
    return results.flat();
  }
} else {
  const result = workerData.map(item => heavyComputation(item));
  parentPort.postMessage(result);
}
```

5.3 集群模式应用

学习目标

- 掌握Cluster模块实现多进程
- 理解主从进程间通信
- 掌握PM2进程管理工具
- 了解sticky session实现

Cluster模式实现

```
const cluster = require('cluster');
const os = require('os');

if (cluster.isPrimary) {
  const cpuCount = os.cpus().length;
  console.log(`Primary ${process.pid} running`);

  for (let i = 0; i < cpuCount; i++) {
    cluster.fork();
  }

  cluster.on('exit', (worker, code, signal) => {
    console.log(`Worker ${worker.process.pid} died. Restarting...`);
    cluster.fork();
  });

  cluster.on('listening', (worker, address) => {
    console.log(`Worker ${worker.process.pid} listening on ${address.port}`);
  });
} else {
  require('./server');
}
```

PM2配置文件

```
// ecosystem.config.js
module.exports = {
  apps: [{
    name: 'my-app',
    script: './dist/main.js',
    instances: 'max',
    exec_mode: 'cluster',
    max_memory_restart: '1G',
    env: {
      NODE_ENV: 'production',
      PORT: 3000,
    },
    logging: {
      log_date_format: 'YYYY-MM-DD HH:mm:ss',
      error_file: './logs/error.log',
      out_file: './logs/out.log',
      merge_logs: true,
    },
  }],
};
```

5.4 缓存策略设计

学习目标

- 掌握多级缓存架构设计
- 理解缓存穿透/击穿/雪崩及解决方案
- 掌握缓存更新策略
- 了解分布式缓存一致性

缓存问题与解决方案

问题	描述	解决方案
缓存穿透	查询不存在的数据，请求直达数据库	布隆过滤器、缓存空值
缓存击穿	热点key过期，大量请求同时打到数据库	互斥锁、永不过期+异步刷新
缓存雪崩	大量key同时过期	随机TTL、多级缓存、限流降级

多级缓存架构

```
class MultilevelCache {
  constructor() {
    this.l1 = new Map();
    this.l2 = redisClient;
    this.l3 = databaseClient;
  }

  async get(key) {
    if (this.l1.has(key)) return this.l1.get(key);

    const l2Data = await this.l2.get(key);
    if (l2Data) {
      this.l1.set(key, l2Data);
      return l2Data;
    }

    const l3Data = await this.l3.findById(key);
    if (l3Data) {
      await this.l2.set(key, l3Data, 3600);
      this.l1.set(key, l3Data);
    }
    return l3Data;
  }

  invalidate(key) {
    this.l1.delete(key);
    return this.l2.del(key);
  }
}
```

Node.js Performance官方指南

<https://nodejs.org/en/docs/guides/simple-profiling>

Node.js性能分析入门指南

Clinic.js

<https://clinicjs.org/>

Node.js性能诊断工具套件，可视化分析

PM2官方文档

<https://pm2.keymetrics.io/docs/usage/quick-start/>

Node.js生产进程管理器完整文档

6 工程化实践（第16-18周）

6.1 代码规范与质量控制

学习目标

- 配置ESLint + Prettier + TypeScript
- 掌握Husky + lint-staged Git钩子
- 理解Commitlint提交规范
- 掌握SonarQube代码质量分析

ESLint配置示例

```
// .eslintrc.json
{
  "root": true,
  "parser": "@typescript-eslint/parser",
  "parserOptions": {
    "ecmaVersion": 2022,
    "sourceType": "module"
  },
  "extends": [
    "eslint:recommended",
    "plugin:@typescript-eslint/recommended",
    "plugin:node/recommended",
    "prettier"
  ],
  "rules": {
    "no-console": ["warn", { "allow": ["warn", "error"] }],
    "@typescript-eslint/no-explicit-any": "error",
    "@typescript-eslint/explicit-function-return-type": "warn",
    "node/no-unsupported-features/es-syntax": "off"
  }
}
```

6.2 单元测试与集成测试

学习目标

- 掌握Jest测试框架的使用
- 理解测试金字塔（单元→集成→E2E）
- 掌握Mock/Stub/Spy技巧
- 了解测试覆盖率与Istanbul

Jest测试示例

```

const UserService = require('../services/UserService');
const UserModel = require('../models/User');

jest.mock('../models/User');

describe('UserService', () => {
  let userService;

  beforeEach(() => {
    userService = new UserService();
    jest.clearAllMocks();
  });

  describe('getUserById', () => {
    it('should return user when found', async () => {
      const mockUser = { id: '1', name: 'Test', email: 'test@test.com' };
      UserModel.findById.mockResolvedValue(mockUser);

      const result = await userService.getUserById('1');

      expect(result).toEqual(mockUser);
      expect(UserModel.findById).toHaveBeenCalledWith('1');
    });

    it('should throw NotFoundError when user not found', async () => {
      UserModel.findById.mockResolvedValue(null);

      await expect(userService.getUserById('999'))
        .rejects.toThrow('用户不存在');
    });
  });
});

```

6.3 CI/CD流程搭建

学习目标

- 掌握GitHub Actions / GitLab CI配置
- 理解CI/CD流水线设计原则
- 掌握自动化测试与部署流程
- 了解蓝绿部署与金丝雀发布

GitHub Actions配置


```

name: CI/CD Pipeline

on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [18.x, 20.x]

    steps:
      - uses: actions/checkout@v4
      - name: Use Node.js ${ matrix.node-version }
        uses: actions/setup-node@v4
        with:
          node-version: ${ matrix.node-version }
          cache: 'pnpm'

      - run: pnpm install --frozen-lockfile
      - run: pnpm lint
      - run: pnpm test --coverage
      - run: pnpm build

  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to production
        run: |
          docker build -t myapp:${ github.sha } .
          docker push myregistry/myapp:${ github.sha }

```

6.4 Docker容器化部署

学习目标

- 掌握Dockerfile编写最佳实践
- 理解多阶段构建优化镜像大小
- 掌握docker-compose编排
- 了解Kubernetes基础概念

多阶段Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json pnpm-lock.yaml ./
RUN corepack enable && pnpm install --frozen-lockfile
COPY . .
RUN pnpm build && pnpm prune --prod

FROM node:20-alpine AS runtime
WORKDIR /app
RUN addgroup -g 1001 -S appgroup && adduser -S appuser -u 1001
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/dist ./dist
COPY --from=builder /app/package.json ./
USER appuser
EXPOSE 3000
CMD ["node", "dist/main.js"]
```

docker-compose编排

```
version: '3.8'
services:
  app:
    build: .
    ports:
      - "3000:3000"
    environment:
      - NODE_ENV=production
      - DATABASE_URL=postgres://user:pass@db:5432/app
      - REDIS_URL=redis://cache:6379
    depends_on:
      - db
      - cache
    restart: unless-stopped

  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: app
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes:
      - pgdata:/var/lib/postgresql/data

  cache:
    image: redis:7-alpine
    volumes:
      - redisdata:/data

volumes:
  pgdata:
  redisdata:
```

Jest官方文档

<https://jestjs.io/docs/getting-started>

JavaScript测试框架完整文档

GitHub Actions文档

<https://docs.github.com/en/actions>

GitHub CI/CD平台完整文档

Docker官方文档

<https://docs.docker.com/>

Docker容器化平台完整文档

ESLint官方文档

<https://eslint.org/docs/latest/>

JavaScript代码检查工具完整文档

7 框架与生态（第19-21周）

7.1 Express框架深入学习

学习目标

- 深入理解Express中间件机制（洋葱模型）
- 掌握路由、错误处理与请求生命周期
- 实现自定义中间件（认证、日志、限流）
- 了解Express性能优化与安全配置

Express中间件架构

```

const express = require('express');
const app = express();

const requestLogger = (req, res, next) => {
  const start = Date.now();
  res.on('finish', () => {
    const duration = Date.now() - start;
    console.log(`${req.method} ${req.path} ${res.statusCode} ${duration}ms`);
  });
  next();
};

const rateLimiter = (maxRequests = 100, windowMs = 60000) => {
  const requests = new Map();
  return (req, res, next) => {
    const key = req.ip;
    const now = Date.now();
    const record = requests.get(key) || { count: 0, startTime: now };

    if (now - record.startTime > windowMs) {
      record.count = 0;
      record.startTime = now;
    }

    record.count++;
    requests.set(key, record);

    if (record.count > maxRequests) {
      return res.status(429).json({ error: '请求过于频繁' });
    }
    next();
  };
};

app.use(express.json());
app.use(requestLogger);
app.use(rateLimiter(100, 60000));

app.use('/api', require('./routes'));

app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(err.status || 500).json({
    code: err.code || 'INTERNAL_ERROR',
    message: err.message,
  });
});

app.listen(3000);

```

7.2 Koa框架深入学习

学习目标

- 理解Koa的async/await中间件模型
- 掌握Koa核心：context/request/response
- 对比Express与Koa的设计哲学差异

- 实现Koa自定义中间件

Express vs Koa对比

特性	Express	Koa
中间件模型	线性 (callback)	洋葱模型 (async/await)
错误处理	回调风格	try/catch
内置功能	丰富 (路由、静态文件等)	极简 (仅HTTP服务)
Context	req/res分离	统一ctx对象
灵活性	约定大于配置	高度可定制
学习曲线	较低	中等

Koa洋葱模型示例

```
const Koa = require('koa');
const app = new Koa();

app.use(async (ctx, next) => {
  console.log('1 - 进入第一层中间件');
  const start = Date.now();
  await next();
  const duration = Date.now() - start;
  ctx.set('X-Response-Time', `${duration}ms`);
  console.log(`1 - 第一层中间件耗时: ${duration}ms`);
});

app.use(async (ctx, next) => {
  console.log('2 - 进入第二层中间件');
  await next();
  console.log('2 - 第二层中间件完成');
});

app.use(async (ctx) => {
  console.log('3 - 处理请求');
  ctx.body = { message: 'Hello Koa' };
});

app.listen(3000);

// 输出顺序: 1进入 → 2进入 → 3处理 → 2完成 → 1完成
```

7.3 NestJS框架深入学习

学习目标

- 掌握NestJS模块/控制器/服务架构
- 理解依赖注入(DI)与控制反转(IoC)
- 掌握NestJS守卫/拦截器/管道

- 了解NestJS微服务传输层

NestJS核心架构

```
import { Module, Controller, Get, Post, Body, Injectable, UseGuards } from '@nestjs/common';

@Injectable()
export class UsersService {
  constructor(
    @InjectRepository(User) private userRepo: Repository<User>,
    private cacheService: CacheService,
  ) {}

  async findById(id: string): Promise<User> {
    const cached = await this.cacheService.get(`user:${id}`);
    if (cached) return cached;

    const user = await this.userRepo.findOne({ where: { id } });
    if (!user) throw new NotFoundException('用户不存在');

    await this.cacheService.set(`user:${id}`, user, 3600);
    return user;
  }
}

@Controller('users')
@UseGuards(AuthGuard)
export class UsersController {
  constructor(private usersService: UsersService) {}

  @Get(':id')
  async findOne(@Param('id') id: string) {
    return this.usersService.findById(id);
  }

  @Post()
  async create(@Body() createUserDto: CreateUserDto) {
    return this.usersService.create(createUserDto);
  }
}

@Module({
  imports: [TypeOrmModule.forFeature([User])],
  controllers: [UsersController],
  providers: [UsersService],
  exports: [UsersService],
})
export class UsersModule {}
```

7.4 中间件开发与微服务架构

学习目标

- 掌握通用中间件设计模式
- 理解微服务通信模式（同步/异步）
- 了解服务注册与发现
- 掌握API网关设计

微服务通信模式

模式	技术	适用场景
同步HTTP	REST/gRPC	简单查询、低延迟要求
异步消息	RabbitMQ/Kafka	事件驱动、解耦服务
服务发现	Consul/Etcd	动态服务注册与发现
API网关	NestJS Gateway/Kong	统一入口、限流、鉴权

Express官方文档

<https://expressjs.com/>

Express框架完整API文档与指南

Koa官方文档

<https://koajs.com/>

Koa框架官方文档

NestJS官方文档

<https://docs.nestjs.com/>

NestJS框架完整文档，包含微服务、GraphQL等

Fastify官方文档

<https://fastify.dev/>

高性能Node.js Web框架

8 高级主题（第22-24周）

8.1 安全最佳实践

学习目标

- 掌握JWT/OAuth2.0认证授权实现
- 理解XSS/CSRF/SQL注入/SSRF等攻击防护
- 掌握helmet/cors/rate-limit等安全中间件
- 了解加密算法与密钥管理

JWT认证实现

```

const jwt = require('jsonwebtoken');
const bcrypt = require('bcrypt');

class AuthService {
  constructor(jwtSecret, jwtExpiry = '7d') {
    this.jwtSecret = jwtSecret;
    this.jwtExpiry = jwtExpiry;
  }

  async hashPassword(password) {
    return bcrypt.hash(password, 12);
  }

  async verifyPassword(password, hash) {
    return bcrypt.compare(password, hash);
  }

  generateToken(payload) {
    return jwt.sign(payload, this.jwtSecret, {
      expiresIn: this.jwtExpiry,
      algorithm: 'HS256',
    });
  }

  verifyToken(token) {
    try {
      return jwt.verify(token, this.jwtSecret, { algorithms: ['HS256'] });
    } catch {
      return null;
    }
  }

  generateTokenPair(payload) {
    const accessToken = this.generateToken(payload);
    const refreshToken = jwt.sign(payload, this.jwtSecret, {
      expiresIn: '30d',
      algorithm: 'HS256',
    });
    return { accessToken, refreshToken };
  }
}

```

安全防护清单

威胁	防护措施	工具/库
XSS	输入过滤、CSP策略、输出编码	helmet、DOMPurify
CSRF	CSRF Token、SameSite Cookie	csurf
SQL注入	预处理语句、ORM	mysql2、Sequelize
暴力破解	限流、账户锁定	express-rate-limit
信息泄露	错误处理、隐藏版本号	helmet hidePoweredBy
中间人攻击	HTTPS、HSTS	helmet、Let's Encrypt

威胁	防护措施	工具/库
依赖漏洞	定期审计、自动更新	npm audit、Snyk

8.2 分布式系统设计

学习目标

- 理解CAP定理与分布式一致性
- 掌握消息队列(RabbitMQ/Kafka)应用
- 了解分布式事务（2PC/Saga模式）
- 掌握分布式锁实现

CAP定理与权衡

策略	保证	牺牲	典型系统
CP	一致性 + 分区容错	可用性	Zookeeper、Etcd
AP	可用性 + 分区容错	强一致性	Cassandra、DynamoDB
CA	一致性 + 可用性	分区容错	单机MySQL（理论）

分布式锁实现

```
class RedisDistributedLock {
  constructor(redisClient) {
    this.redis = redisClient;
  }

  async acquire(key, ttl = 10000, retryCount = 3, retryDelay = 200) {
    const token = crypto.randomUUID();

    for (let i = 0; i < retryCount; i++) {
      const result = await this.redis.set(key, token, 'PX', ttl, 'NX');
      if (result === 'OK') return token;

      await new Promise(resolve => setTimeout(resolve, retryDelay));
    }

    throw new Error(`获取锁失败: ${key}`);
  }

  async release(key, token) {
    const script = `
      if redis.call("get", KEYS[1]) == ARGV[1] then
        return redis.call("del", KEYS[1])
      else
        return 0
      end
    `;
    return this.redis.eval(script, 1, key, token);
  }
}
```

8.3 服务监控与日志系统

学习目标

- 掌握Winston/Pino结构化日志
- 理解APM（应用性能监控）原理
- 掌握Prometheus + Grafana监控体系
- 了解分布式链路追踪（Jaeger/Zipkin）

结构化日志系统

```

const winston = require('winston');

const logger = winston.createLogger({
  level: process.env.LOG_LEVEL || 'info',
  format: winston.format.combine(
    winston.format.timestamp({ format: 'YYYY-MM-DD HH:mm:ss.SSS' }),
    winston.format.errors({ stack: true }),
    winston.format.json(),
  ),
  defaultMeta: {
    service: 'my-app',
    version: process.env.APP_VERSION || '1.0.0',
    environment: process.env.NODE_ENV,
  },
  transports: [
    new winston.transports.Console({
      format: winston.format.combine(
        winston.format.colorize(),
        winston.format.printf(({ timestamp, level, message, ...meta }) => {
          return `${timestamp} [${level}]: ${message} ${Object.keys(meta).length ? JSON.stringify(meta) : ''}`;
        })
      ),
    }),
    new winston.transports.File({
      filename: 'logs/error.log',
      level: 'error',
      maxsize: 10485760,
      maxFiles: 5,
    }),
    new winston.transports.File({
      filename: 'logs/combined.log',
      maxsize: 10485760,
      maxFiles: 10,
    }),
  ],
});

class RequestLogger {
  static logRequest(req, res, duration) {
    logger.info('HTTP Request', {
      method: req.method,
      path: req.path,
      status: res.statusCode,
      duration,
      ip: req.ip,
      userAgent: req.get('user-agent'),
      userId: req.user?.id,
    });
  }
}

```

Prometheus指标采集

```
const client = require('prom-client');

const register = new client.Registry();
client.collectDefaultMetrics({ register });

const httpRequestsTotal = new client.Counter({
  name: 'http_requests_total',
  help: 'Total number of HTTP requests',
  labelNames: ['method', 'path', 'status'],
  registers: [register],
});

const httpRequestDuration = new client.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'path', 'status'],
  buckets: [0.1, 0.5, 1, 2, 5],
  registers: [register],
});

function metricsMiddleware(req, res, next) {
  const start = Date.now();
  res.on('finish', () => {
    const duration = (Date.now() - start) / 1000;
    const labels = { method: req.method, path: req.route?.path || req.path, status: res.statusCode };
    httpRequestsTotal.inc(labels);
    httpRequestDuration.observe(labels, duration);
  });
  next();
}
```

8.4 Node.js与前端工程结合

学习目标

- 掌握SSR(Next.js/Nuxt.js)原理与实现
- 了解BFF(Backend For Frontend)架构
- 掌握Webpack/Vite与Node.js的结合
- 了解Isomorphic/Universal JavaScript

BFF架构设计

```

class BFFRouter {
  constructor() {
    this.aggregators = new Map();
  }

  register(path, aggregator) {
    this.aggregators.set(path, aggregator);
  }

  async handle(path, params, userContext) {
    const aggregator = this.aggregators.get(path);
    if (!aggregator) throw new Error(`路由不存在: ${path}`);

    const results = {};
    const tasks = Object.entries(aggregator.sources).map(async ([key, config]) => {
      if (config.requiredRoles && !config.requiredRoles.some(r => userContext.roles.includes(r))) {
        results[key] = { error: '无权限访问' };
        return;
      }
      try {
        results[key] = await this.fetchFromService(config, params, userContext);
      } catch (error) {
        if (config.required) throw error;
        results[key] = null;
      }
    });

    await Promise.all(tasks);
    return aggregator.transform(results);
  }
}

```

OWASP Node.js安全指南

https://cheatsheetseries.owasp.org/cheatsheets/Nodejs_Security_Cheat_Sheet.html

OWASP官方Node.js安全实践清单

Prometheus Node.js客户端

<https://github.com/siimon/prom-client>

Node.js Prometheus指标采集库

Winston官方文档

<https://github.com/winstonjs/winston>

Node.js多传输日志库

Next.js官方文档

<https://nextjs.org/docs>

React SSR框架完整文档



实践项目清单

以下12个实战项目覆盖所有核心模块，建议按顺序完成。每个项目都对应特定技能点，完成后可独立运行展示。

序号	项目名称	对应模块	核心技能点	难度
1	异步任务调度器	模块1	事件循环、Promise、并发控制	☆☆
2	大文件日志分析器	模块2	Stream、Buffer、文件系统	☆☆☆
3	实时协作白板	模块3	WebSocket、TCP、HTTP	☆☆☆
4	电商订单系统后端	模块4	MySQL事务、MongoDB、Redis缓存	☆☆☆☆
5	高并发短链服务	模块5	缓存策略、Cluster、性能优化	☆☆☆☆
6	全自动化部署API服务	模块6	Docker、CI/CD、测试	☆☆☆☆
7	微服务架构博客平台	模块7	NestJS、中间件、微服务通信	☆☆☆☆☆
8	全栈电商平台	模块8	SSR、BFF、安全、监控	☆☆☆☆☆
9	CLI脚手架工具	模块2	文件操作、命令行交互、模板引擎	☆☆☆
10	API网关服务	模块7	反向代理、限流、服务发现	☆☆☆☆
11	实时消息推送系统	模块3+8	WebSocket、Redis Pub/Sub、集群	☆☆☆☆
12	分布式任务调度平台	模块5+8	分布式锁、消息队列、监控	☆☆☆☆☆

项目1：异步任务调度器 详细设计


```

class TaskScheduler {
  constructor(concurrency = 5) {
    this.concurrency = concurrency;
    this.running = 0;
    this.queue = [];
  }

  schedule(taskFn, priority = 0) {
    return new Promise((resolve, reject) => {
      this.queue.push({ taskFn, resolve, reject, priority });
      this.queue.sort((a, b) => b.priority - a.priority);
      this.run();
    });
  }

  async run() {
    while (this.running < this.concurrency && this.queue.length > 0) {
      const { taskFn, resolve, reject } = this.queue.shift();
      this.running++;

      try {
        const result = await taskFn();
        resolve(result);
      } catch (error) {
        reject(error);
      } finally {
        this.running--;
        this.run();
      }
    }
  }
}

const scheduler = new TaskScheduler(3);

async function fetchData(url) {
  const res = await fetch(url);
  return res.json();
}

const results = await Promise.all([
  scheduler.schedule(() => fetchData('/api/users'), 2),
  scheduler.schedule(() => fetchData('/api/orders'), 1),
  scheduler.schedule(() => fetchData('/api/products'), 1),
]);

```



常见面试题解析

一、Node.js核心原理

Q1: Node.js的事件循环和浏览器的事件循环有什么区别？

答：主要区别在于：

1. Node.js基于libuv实现，有6个阶段（timers → pending → idle/prepare → poll → check → close），浏览器只有宏任务和微任务两个队列
2. Node.js中process.nextTick优先级高于Promise微任务
3. Node.js的微任务在每个阶段切换时执行，浏览器在每个宏任务后执行
4. Node.js有setImmediate（check阶段），浏览器没有

Q2: 什么是Node.js的Reactor模式？

答：Reactor模式是Node.js异步I/O的核心设计模式。它由以下组件构成：

1. **资源**：I/O操作的来源（文件、网络等）
2. **事件多路分解器**：libuv的epoll/kqueue/IOCP，负责监听多个I/O事件
3. **事件队列**：存储已就绪的事件
4. **事件循环**：从队列中取出事件，分发给对应的回调函数
5. **请求处理器**：应用程序定义的回调函数

Q3: V8的垃圾回收机制是怎样的？

答：V8采用分代式垃圾回收：

1. **新生代**（1-8MB）：使用Scavenge算法，将堆分为From和To两个半空间，存活对象从From复制到To，然后交换
2. **老生代**：使用Mark-Sweep（标记清除）和Mark-Compact（标记整理），还引入增量标记（Incremental Marking）减少停顿
3. **晋升条件**：经历过一次Scavenge回收或To空间使用超过25%的对象会被移到老生代

二、异步编程

Q4: Promise.all和Promise.allSettled的区别？

答：

- **Promise.all**：所有Promise都fulfilled才fulfilled，任一rejected就rejected（快速失败）
 - **Promise.allSettled**：等待所有Promise都settled（无论fulfilled还是rejected），返回每个Promise的状态和值
- 实际应用中，批量请求推荐使用allSettled避免单个失败导致整体失败。

Q5: 如何实现一个并发控制的异步任务执行器?

答: 核心思路是维护一个运行计数器和等待队列, 当运行数小于并发限制时从队列取出任务执行:

```
async function asyncPool(limit, items, iteratorFn) {
  const results = [];
  const executing = new Set();

  for (const [index, item] of items.entries()) {
    const p = Promise.resolve().then(() => iteratorFn(item, index));
    results.push(p);
    executing.add(p);
    p.finally(() => executing.delete(p));

    if (executing.size >= limit) {
      await Promise.race(executing);
    }
  }

  return Promise.all(results);
}
```

三、网络编程

Q6: HTTP/1.1、HTTP/2和HTTP/3的主要区别?

答:

- **HTTP/1.1**: 文本协议、队头阻塞、需要多个TCP连接实现并发
- **HTTP/2**: 二进制帧、多路复用 (单连接并发)、头部压缩 (HPACK)、服务器推送
- **HTTP/3**: 基于QUIC (UDP)、解决了TCP层的队头阻塞、0-RTT连接建立、内置TLS 1.3

Q7: 如何设计一个高可用的WebSocket集群?

答: 核心问题是多节点间的消息同步:

1. 使用Redis Pub/Sub作为消息总线, 节点间通过Redis转发消息
2. 使用Sticky Session确保同一用户连接到同一节点
3. 设计房间(Room)概念, 将用户分组管理
4. 实现心跳检测和断线重连机制
5. 使用消息确认机制确保消息不丢失

四、性能优化

Q8: 如何排查Node.js内存泄漏?

答: 排查步骤:

1. 使用process.memoryUsage()监控内存趋势
2. 使用v8.getHeapSnapshot()生成堆快照
3. 使用Chrome DevTools的Memory面板对比快照
4. 关注Retained Size最大的对象
5. 检查Dominator Tree找到GC根引用链
6. 常见原因: 全局变量、闭包引用、事件监听器未移除、定时器未清理

Q9: Cluster模式和Worker Threads的区别与选择?

答:

- **Cluster**: 多进程, 每个进程独立V8实例和内存空间, 进程间通过IPC通信, 适合利用多核CPU处理HTTP请求
- **Worker Threads**: 多线程, 共享ArrayBuffer, 通信开销更小, 适合CPU密集型计算任务
- 选择原则: 网络服务用Cluster, 计算任务用Worker Threads

五、安全

Q10: JWT和Session认证的优缺点?

答:

- **JWT**: 无状态、可跨域、自包含信息; 但无法主动失效、token较大、刷新机制复杂
- **Session**: 服务端可控、可即时失效、token小; 但需服务端存储、跨域困难、扩展性差
- 推荐方案: Access Token(JWT短期) + Refresh Token(长期) + Redis黑名单

六、系统设计

Q11: 如何设计一个支持百万并发的短链服务?

答:

1. **短链生成**: 分布式ID生成器 (Snowflake) + Base62编码
2. **存储**: MySQL持久化 + Redis缓存热点短链
3. **读取**: Redis缓存命中直接302跳转, 未命中回源MySQL
4. **高可用**: Cluster多进程 + Nginx负载均衡 + 读写分离
5. **防刷**: 布隆过滤器防恶意短链 + 限流 + 黑名单
6. **监控**: 访问统计 + Prometheus指标 + 告警

Q12: 什么是Saga模式? 如何实现分布式事务?

答: Saga模式将长事务拆分为多个本地事务, 每个本地事务有对应的补偿操作:

1. **编排式(Choreography)**: 各服务通过事件通信, 无中心协调器, 松耦合但难追踪
2. **协调式(Orchestration)**: 中央协调器控制流程, 集中管理但耦合度高
3. Node.js实现: 可使用消息队列 (RabbitMQ) + 补偿事务 + 状态机
4. 注意幂等性设计, 确保补偿操作可重复执行



推荐学习资源汇总

官方文档与网站

资源名称	网址	说明
Node.js官网	https://nodejs.org/	官方下载、文档入口
Node.js API 文档	https://nodejs.org/docs/latest/api/	核心模块 API参考
Node.js Best Practices	https://github.com/goldbergonyi/nodebestpractices	社区最佳实践合集
Express官方文档	https://expressjs.com/	Express框架文档
Koa官方文档	https://koajs.com/	Koa框架文档
NestJS官方文档	https://docs.nestjs.com/	NestJS框架文档
Fastify官方文档	https://fastify.dev/	高性能Web框架文档
TypeScript 官方文档	https://www.typescriptlang.org/docs/	TypeScript 语言文档
MongoDB Node.js Driver	https://www.mongodb.com/docs/drivers/node/current/	MongoDB 驱动文档
Redis ioredis	https://github.com/redis/ioredis	Redis客户端文档
MySQL2	https://github.com/sidorares/node-mysql2	MySQL客户端文档
PostgreSQL pg	https://node-postgres.com/	PostgreSQL 客户端文档
TypeORM	https://typeorm.io/	TypeScript ORM文档

资源名称	网址	说明
Sequelize	https://sequelize.org/	Node.js ORM文档
Socket.IO	https://socket.io/docs/v4/	WebSocket 框架文档
ws	https://github.com/websockets/ws	WebSocket 库文档
Jest	https://jestjs.io/docs/getting-started	测试框架文档
Docker	https://docs.docker.com/	Docker容器化文档
PM2	https://pm2.keymetrics.io/docs/usage/quick-start/	进程管理器文档
Clinic.js	https://clinicjs.org/	性能诊断工具
Next.js	https://nextjs.org/docs	React SSR 框架文档
OWASP Node.js安全	https://cheatsheetseries.owasp.org/cheatsheets/Nodejs_Security_Cheat_Sheet.html	安全实践清单
GitHub Actions	https://docs.github.com/en/actions	CI/CD平台文档
ESLint	https://eslint.org/docs/latest/	代码检查工具文档
Prometheus prom-client	https://github.com/siimon/prom-client	指标采集库文档
Winston	https://github.com/winstonjs/winston	日志库文档

推荐书籍

书名	作者	重点章节
《Node.js设计模式（第3版）》	Mario Casciaroma	设计模式、流、集群、微服务
《深入浅出Node.js》	朴灵	事件循环、异步编程、内存管理
《Node.js实战（第2版）》	Alex Young等	项目实战、Express、数据库
《Node.js硬实战：核心模块与原生API》	潘俊昌	核心模块深入、网络编程

书名	作者	重点章节
《分布式系统设计》	Zhuoqing Morley Mao	CAP、一致性、分布式事务

推荐在线课程

平台	课程名称	网址
Udemy	Node.js - The Complete Guide	https://www.udemy.com/course/nodejs-the-complete-guide/
Frontend Masters	Production-Grade Node.js	https://frontendmasters.com/courses/production-node/
Pluralsight	Node.js Advanced	https://www.pluralsight.com/paths/nodejs-advanced
慕课网	Node.js高级编程	https://www.imooc.com/

技术博客与社区

名称	网址	说明
Node.js Blog	https://nodejs.org/en/blog	官方博客，版本发布说明
Node Weekly	https://nodeweekly.com/	每周Node.js新闻邮件
Dev.to Node.js	https://dev.to/t/node	开发者社区Node.js话题
Stack Overflow	https://stackoverflow.com/questions/tagged/node.js	问答社区
掘金	https://juejin.cn/tag/Node.js	中文技术社区

💡 学习建议

1. **以项目驱动学习**：每学完一个模块立即完成对应实战项目
2. **阅读源码**：选择Express/Koa等框架的核心源码进行阅读
3. **写技术博客**：将学习心得整理成博客，加深理解
4. **参与开源**：为Node.js生态项目提交PR，提升工程能力
5. **模拟面试**：每周至少练习5道面试题，培养系统设计思维
6. **持续关注**：订阅Node.js Blog和Node Weekly，跟进最新动态

● Node.js P6高级工程师学习计划 | 总周期24周 | 8大核心模块 | 12个实战项目

坚持学习，持续精进，祝早日达到P6技术水平！